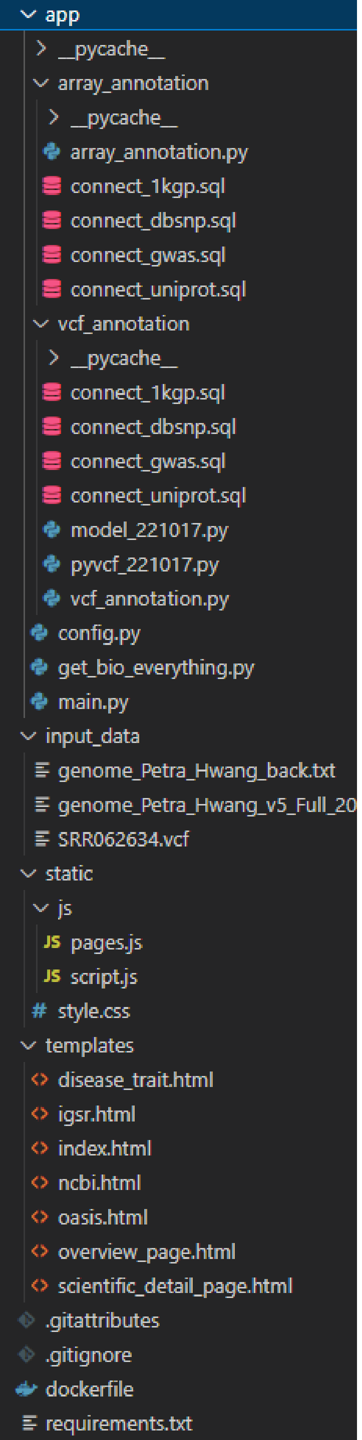
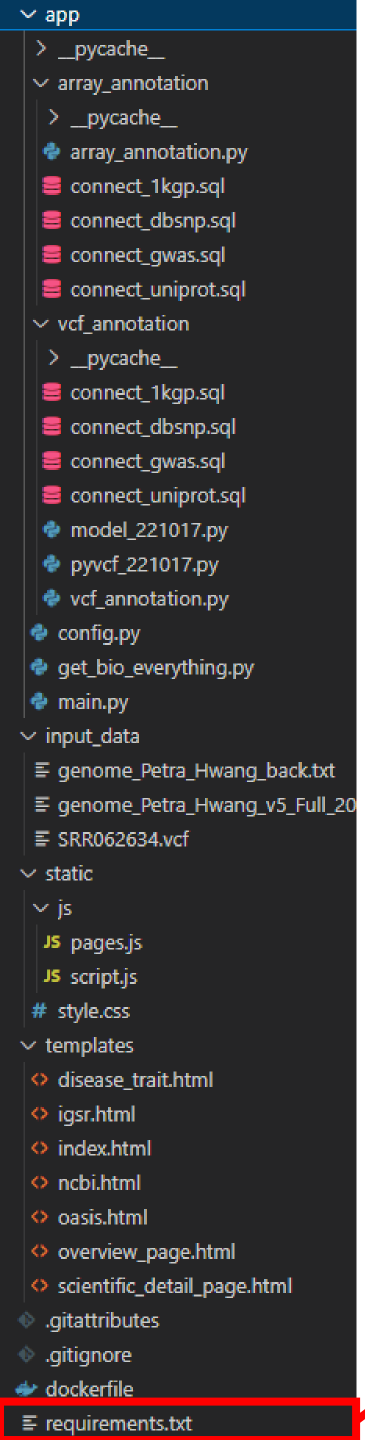


BIO_Process

0. requirements.txt
1. 데이터 가공 및 적재 (DB Load)
2. VCF에서 최종 테이블 (VCF to Final Table)
3. Array Data Format에서 최종 테이블 (Array Data Format to Final Table)
4. 웹화면 (FastAPI)
5. Docker활용 서버 배포

0. requirements.txt




```
fastapi==0.95.1
greenlet==2.0.2
h11==0.14.0
idna==3.4
Jinja2==3.1.2
MarkupSafe==2.1.2
numpy==1.24.3
pandas==2.0.1
pydantic==1.10.7
PyMySQL==1.0.3
python-dateutil==2.8.2
python-multipart==0.0.6
pytz==2023.3
six==1.16.0
sniffio==1.3.0
SQLAlchemy==2.0.10
starlette==0.26.1
tqdm==4.65.0
typing-extensions==4.5.0
tzdata==2023.3
uvicorn==0.21.1
```

1. 가상환경 (venv) 에서
pip freeze > requirement.txt

1. 데이터 가공 및 적재 (DB Load)

DB Load

단계	내용	사용 언어	DB 테이블명
dbSNP 데이터베이스 구축	1) Download (https://ftp.ncbi.nlm.nih.gov/snp/) 2) Chromosome 별로 분리 (dbsnp.sh 활용) 3) ['CHR'-'POS'-'REF'-'ALT'] 형태의 key 값 컬럼 추가	리눅스 Python SQL	grch37_dbsnp_chr{chrom}
GWAS Catalog 데이터베이스 구축	1) Download (http://ftp.ebi.ac.uk/pub/databases/gwas/) 2) 'RISK_ALLELE' 컬럼 생성 by 'STRONGEST_SNP' 3) 'OR' 필드 참조하여 'risk_type' 컬럼 생성 (OR 또는 BETA)	SQL	gwas_catelog
1KGP 데이터베이스 구축	인구집단 별 대립인자 빈도(allele frequency) 계산	Python	1000genome_chr{chrom}
uniprot 데이터베이스 구축	coordinate 열의 내용을 가공하여 새로운 열 추가하여 DB 적재	Python	uniprot
ncbi 데이터베이스 구축	1) ncbi_crawl.py를 통해 크롤링한 데이터를 DB 적재 2) 쿼리문으로 공통적으로 생기는 필요없는 row를 삭제한다	Python SQL	ncbi
medlineplus 데이터베이스 구축	1) category_crawl.py를 통해 category를 먼저 크롤링한 후 2) medlineplus_crawl.py의 input을 category로 하여 크롤링한 데이터를 DB 적재	Python	medlineplus
한의학 데이터베이스 구축	PharmDB-K 데이터베이스 활용 Disease/Trait 연관 한의학 탐색	SQL	pharmdbk_tkm>pharmdbk oasis 테이블들 > oasis tkm_ref_tb>특수제작
의약품 데이터베이스 구축	의약품안전나라 활용 한약재 연관 의약품 목록 확보	SQL	medicine >식약처

1. **dbSNP** > grch37_dbsnp_chr{chrom_id}
    I. dbsnp 텍스트 파일을 리눅스 sed명령어를 활용하여 chrom기준으로 25개의 텍스트 파일로 분리한다.
    II. python으로 만든 rsID_ALT값과 connect_key값을 열 추가한 dataframe을 DB에 적재한다.
2. **gwas_catelog** > gwas_catelog
    I. gwas_catelog 텍스트 파일을 파싱하여 db에 입력한다.
    II. gwas_catelog_tb에서 STRONGEST_SNP 필드값의 정보를 substring_index() 쿼리문으로 세미콜론을 기준으로 분리하고 STRONGEST_SNP 의 risk_allele, rsID을 column 추가하여 테이블을 저장한다.
3. **1KGP** > 1000genome_chr{chrom_id}
    I. python으로 인종별 합산 계산식을 거친 dataframe을 DB에 적재한다. >> 1000genom_chr1~22
4. **UNIPROT** > uniprot
    I. python으로 coordinate열을 가공하여 열 추가한 dataframe을 DB에 적재한다. >> uniport
5. **NCBI** > ncbi
    I. ncbi_crawl.py를 통해 크롤링한 데이터를 DB 적재
    II. 쿼리문으로 공통적으로 생기는 필요없는 row를 삭제한다 >> ncbi
6. **Medlineplus** > medlineplus
    I. category_crawl.py를 통해 category를 먼저 크롤링한 후
    II. medlineplus_crawl.py의 input을 category로 하여 크롤링한 데이터를 DB 적재 >> medlineplus
7. **한의학** > pharmdbk_tkm_relation & oasis_clinical & oasis_preclinical & tkm_ref_tb
    I. pharmdbk 제공하는 데이터 DB 적재 (손수 크롤링)
    II. oasis 제공하는 데이터 DB 적재 (손수 크롤링)
8. **의약품 데이터** > medicine
    I. 식약처 활용한 DB 적재 (손수 크롤링)

2.dbsnp (raw : ncbi)

insert_dbsnp.py
run_sh.py
split.sh

raw_data

grch37_dbsnp.vcf
유형: vCalendar 파일
크기: 114GB
수정한 날짜: 2022-11-16 오전 10:37

split.sh

```
sed -n -e '/^Y  /p' hg38_avsnp150.txt >> y.csv
sed -n -e '/^MT /p' hg38_avsnp150.txt >> mt.csv

split -l 10000000 1.csv 1_
split -l 10000000 2.csv 2_
```

sed로 chrom_id별 자르기
split으로 db적재 가능한 크기씩 자르기

insert_dbsnp.py

```
chunk_df = pd.read_csv(FILE_PATH, chunksize=1000000, header=None, names=['CHR','POS','rsID','REF','ALT','QUAL','FILTER','INFO'], sep='\t'
for dbsnp_df in chunk_df:
```

```
dbsnp_df['rsID_ALT'] = dbsnp_df["rsID"]+"-"+dbsnp_df["ALT"]
dbsnp_df['connect_key'] = dbsnp_df["CHR"].astype(str)+'-'+dbsnp_df["POS"].astype(str)+'-'+dbsnp_df["REF"].astype(str)+'-'+dbsnp_df["ALT"].astype(str)
dbsnp_df.to_sql(name=f"{i}", con=engine, if_exists = 'append', index = False)
```

rsID_ALT 와 connect_key를 추가한 dataframe을 DB적재

2.dbsnp

원본 데이터와의 차이점

CHR	POS	rsID	REF	QUAL	FILTER	INFO	ALT	rsID_ALT	connect_key
1	10019	rs775809821	TA	.	.	RS=775809821;RSPOS=10020;dbSNPBuildID=14...	T	rs775809821-T	1-10019-TA-T
1	10039	rs978760828	A	.	.	RS=978760828;RSPOS=10039;dbSNPBuildID=15...	C	rs978760828-C	1-10039-A-C
1	10043	rs1008829651	T	.	.	RS=1008829651;RSPOS=10043;dbSNPBuildID=1...	A	rs1008829651-A	1-10043-T-A
1	10051	rs1052373574	A	.	.	RS=1052373574;RSPOS=10051;dbSNPBuildID=1...	G	rs1052373574-G	1-10051-A-G

7	rsID_ALT
8	connect_key

gwas join하는 key값

vcf join하는 key

dbsnp의 데이터를 그대로 가져오지 않고 python에서 로직을 더하여
rsID_ALT, connect_key를 추가하였다.

rsID_ALT : rsID, ALT 열을 나열한 값으로 join을 위한 key값 추가
connect_key : chrom, pos, ref, alt 열을 나열한 값으로 join을 위한 key값 추가

3.gwas_catelog (raw : embl-ebi)

make_gwas_catelog_numbers_tb.sql
make_gwas_catelog_row_tb.sql
make_gwas_catelog_separate_tb.sql

raw_data

gwas_catalog_v1.0-associations_e107_r2... 2022-08-04

gwas_catalog_v1.0-associations_e107_r2022-07-30.tsv
유형: TSV 파일
크기: 200MB

make_row

```
INSERT INTO __bio_db.gwas_catelog_row_tb
SELECT b.number_idx,a.* from gwas_catalog_raw_tb AS a
join gwas_catelog_numbers_tb AS b
on CHAR_LENGTH(STRONGEST_SNP) - char_length(REPLACE(STRONGEST_SNP, ';', '')) >= number_idx - 1
;
```

strongest_snp가 여러개인 경우 개수만큼 열을 늘림

make_sep

```
INSERT INTO __bio_db.gwas_catelog_tb
SELECT
```

•
•
•

```
trim(substring_index(substring_index(STRONGEST_SNP, ';', number_idx),';',-1)) AS STRONGEST_SNP,
trim(substring_index(substring_index(substring_index(STRONGEST_SNP, ';', number_idx),';',-1),'-',-1)) AS SNPS,
trim(substring_index(substring_index(substring_index(STRONGEST_SNP, ';', number_idx),';',-1),'-',1)) AS SNPS,
from __bio_db.gwas_catelog_row_tb AS a
```

열을 늘린 상태에서 하나씩 잘라냄

3.gwas_catelog

원본 데이터와의 차이점

DATE_ADDED_TO_CATALOG	PUBMEDID	AUTHOR	DATE	JOURNAL	LINK	STUDY	DISEASE_TRAIT
2009-12-14	19915573	Asano K	2009-11-15	Nat Genet	www.ncbi.nlm.nih.gov/pubmed/19915573	A genome-wide association study identifies three ...	Ulcerative colitis
2009-12-14	19915573	Asano K	2009-11-15	Nat Genet	www.ncbi.nlm.nih.gov/pubmed/19915573	A genome-wide association study identifies three ...	Ulcerative colitis

INIT_SAMP_SIZE	REP_SAMP_SIZE	REGION	CHR_ID	CHR_POS	REPORTED_GENE	MAPPED_GENE	UPSTREAM_GENE_ID	DOWNSTREAM_GENE_ID
376 Japanese ancestry cases, ...	376 Japanese ancestry ...	1q23.3	1	161509955	FCGR2A	FCGR2A		
376 Japanese ancestry cases, ...	376 Japanese ancestry ...	6p21.33	6	31143579	HLA	CCHCR1, CCHCR1, CCH...		

SNP_GENE_IDS	UPSTREAM_DIST	DOWNSTREAM_DIST	STRONGEST_SNP	EFFECT_ALLELE	SNPS	MERGED	SNP_ID_CURRENT	var_type	INTERGENIC	RISK_AF	PVALUE	PVALUE_MLOG
ENSG00000143226			rs1801274-?	?	rs1801274	0	1801274	missense_variant	0	0.78	2E-12	11.699
ENSG00000204536, EN...			rs9263739-T	T	rs9263739	0	9263739	intron_variant	0	0.15	4E-67	66.3979

PVALUE_TEXT	OR	CI	PLATFORM	CNV
	1.59	[1.39-1.82]	Illumina [513923]	N
	2.73	[2.43-3.07]	Illumina [513923]	N

22	STRONGEST_SNP

rs1015362-G; rs4911414-T
rs1015362-G; rs4911414-T
rs1015362-G; rs4911414-T
rs35264875-T
rs1015362-G; rs4911414-T

rs1015362-G
rs4911414-T
rs1015362-G
rs4911414-T
rs1015362-G
rs35264875-T
rs4911414-T

gwas_catelog의 데이터의 STRONGEST_SNP열에서
여러값이 ;을 기준으로 합쳐져 있는 부분을 행단위로 분리함

4.1KGP (raw : rgsr)

main.py
split_cat.sh

raw_data

chr1.vcf
유형: vCalendar 파일
크기: 61.1GB

split.sh

```
split -l  1000000 [파일명]
```

split으로 db적재 가능한 크기씩 자르기

main.py

```
append_columns(999,1001)
ACB_df = raw_vcf.iloc[:,sum(columns,[])]
ACB_sum_df = sum_gt("ACB",ACB_df,ACB_count)
# ACB_df.to_csv(f"result/b.csv", mode ='a', sep=',', index = False)
# ACB_sum_df.to_csv(f"result/c.csv", mode ='a', sep=',', index = False)
print("1/26")
```

26개로 구분된 인종들을 각각 계산식을 거쳐서
dataframe으로 묶고 DB적재

4.1KGP (1000genome_chr{chrom_id})

원본 데이터와의 차이점

CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO	FORMAT	connect_key	ACB	ASW
1	10177	.	A	AC	100	PASS	AC=2130;AF=0.425...	GT	1-10177-A-AC	{'0': 0.5833333333333334, '1': 0.41666666666...	{'0': 0.6557377049180327, '1': 0.34426229508...
1	10235	.	T	TA	100	PASS	AC=6;AF=0.001198...	GT	1-10235-T-TA	{'0': 1.0}	{'0': 1.0}
1	10352	.	T	TA	100	PASS	AC=2191;AF=0.437...	GT	1-10352-T-TA	{'0': 0.546875, '1': 0.453125}	{'0': 0.6065573770491803, '1': 0.39344262295...
1	10505	.	A	T	100	PASS	AC=1;AF=0.000199...	GT	1-10505-A-T	{'0': 0.9947916666666666, '1': 0.00520833333	{'0': 1.0}

1KGP의 데이터를 그대로 가져오지 않고 python에서 로직을 더하여 connect_key, 26개의 인종별 분포도을 추가하였다.

connect_key : chrom, pos, ref, alt 열을 나열한 값으로 join을 위한 key값 추가
인종별 분포도 : 딕셔너리 형태로 묶어서 한인종당 한열에 요약하여 추가

5.uniport (raw : embl-ebi)

main.py
make_tb.py
sed_split.sh

raw_data

homo_sapiens_variation.txt
유형: 텍스트 문서
크기: 5.92GB
수정한 날짜: 2022-12-15 오

split.sh

```
# 설명 삭제
sed -i '1,/___________/ d' homo_sapiens_variation.txt> a.txt

# split
split -l  5000000 homo_sapiens_variation.txt homo_
```

raw_data의 설명부분을 제거하기위한 sed명령어와
db적재 가능한 크기씩 자르기위한 split명령어

main.py

```
coordinate_df = raw_df.loc[:,"CHROMOSOME COORDINATE"]
uniprot_Allele_list = list()
for row in coordinate_df:
    row = str(row)
    uniprot_Allele = row[-3:]
    if '>' in uniprot_Allele:
        uniprot_Allele_list.append(uniprot_Allele.split('>'))
    else:
        uniprot_Allele_list.append(['PASS','PASS'])
allele_df = pd.DataFrame(uniprot_Allele_list, columns=['UNIPROT_REF','UNIPROT_ALT'])
new_df = pd.concat([raw_df,allele_df], axis=1)
```

chromosome_coordinate열을 참조하여 uniprot_ref와 uniport_alt을 생성한 dataframe으로 DB적재

5.uniprot (uniprot)

원본 데이터와의 차이점

CHROMOSOME COORDINATE	ENSEMBL GENE ID	ENSEMBL TRANSCRIPT ID	ENSEMBL TRANSLATION ID	EVIDENCE	UNIPROT_REF	UNIPROT_ALT
NC_000017.11:g.30172593A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	gnomAD,ClinGen	A	G
NC_000017.11:g.30178074A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,ClinGen	A	G
NC_000017.11:g.30178074A>C	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,ClinGen	A	C
NC_000017.11:g.30178078T>C	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,ClinGen	T	C
NC_000017.11:g.30178091G>C	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,gnomAD,ClinGen	G	C
NC_000017.11:g.30178090A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	gnomAD,ClinGen	A	G
NC_000017.11:g.30178099C>T	ENSG00000126653	ENST00000612959	ENSP00000477862	1000Genomes,TOPMed,gnomAD,ClinGen	C	T
NC_000017.11:g.30178098G>A	ENSG00000126653	ENST00000612959	ENSP00000477862	gnomAD,ClinGen	G	A
NC_000017.11:g.30178102A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,ClinGen	A	G
NC_000017.11:g.30178106T>G	ENSG00000126653	ENST00000612959	ENSP00000477862	ExAC,gnomAD,ClinGen	T	G
NC_000017.11:g.30178107G>A	ENSG00000126653	ENST00000612959	ENSP00000477862	TOPMed,gnomAD,ClinGen	G	A
NC_000017.11:g.30178110A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	ExAC,gnomAD,ClinGen	A	G
NC_000017.11:g.30178117A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	ExAC,gnomAD,ClinGen	A	G
NC_000017.11:g.30178117A>T	ENSG00000126653	ENST00000612959	ENSP00000477862	ExAC,gnomAD,ClinGen	A	T
NC_000017.11:g.30178122T>C	ENSG00000126653	ENST00000612959	ENSP00000477862	ExAC,gnomAD,ClinGen	T	C
NC_000017.11:g.30178127C>G	ENSG00000126653	ENST00000612959	ENSP00000477862	gnomAD,ClinGen	C	G
NC_000017.11:g.30178128A>G	ENSG00000126653	ENST00000612959	ENSP00000477862	gnomAD,ClinGen	A	G
NC_000017.11:g.301781[illegible]C>A	ENSG00000126653	ENST00000612959	ENSP00000477862	NCI-TCGA,ExAC,gnomAD,ClinGen	C	A

15	UNIPROT_REF
16	UNIPROT_ALT

uniprot의 데이터의 CHROMOSOME COORDINATE열에서
변이를 나타내는 마지막 3개값이 >을 기준으로 합쳐져 있는 부분을 분리함

6.ncbi

delete_row.sql
genome_id_list
ncbi_crawl.py

raw_data https://ncbi.nlm.nih.gov/gene/{genome_id}

ncbi_crawl.py

```
summary_dict_list = list()
print(f'process : {genome_id_list.index(genome_id)+1}/{len(genome_id_list)} genome_id : {genome_id}')

url_requests = requests.get(f'https://ncbi.nlm.nih.gov/gene/{genome_id}')
soup = BeautifulSoup(url_requests.content,'html.parser')

summary_div = soup.find('div', {'id':'summaryDiv'}).find('dl')

table_title_list = summary_div.find_all('dt')
table_content_list = summary_div.find_all('dd')

for i in range(len(table_title_list)):
    temp_dict = dict()
    temp_dict['genome_id'] = genome_id
    temp_dict['title'] = table_title_list[i].get_text().replace('\n','').replace('\t','').replace('  ','').strip()
    temp_dict['content'] = table_content_list[i].get_text().replace('\n','').replace('\t','').replace('  ','').strip()
    summary_dict_list.append(temp_dict)
summary_df = pd.DataFrame(summary_dict_list)

summary_df.to_sql(name='ncbi', con=db_connection, if_exists='append', index=False)
print(error_list)
```

delete_row.sql

```
delete from ncbi where title = 'NEW';
```

불규칙적으로 발생하는 필요없는 데이터를 제거하는 가공 작업

6.ncbi (ncbi & ncbi_ref)

genome_id	title	content
1	Official Symbol	A1BGprovided by HGNC
1	Official Full Name	alpha-1-B glycoproteinprovided by HGNC
1	Primary source	HGNC:HGNC:5
1	See related	Ensembl:ENSG00000121410MIM:138670;Alliance...
1	Gene type	protein coding
1	RefSeq status	REVIEWED
1	Organism	Homo sapiens
1	Lineage	Eukaryota; Metazoa; Chordata; Craniata; Vertebra...
1	Also known as	A1B; ABG; GAB; HYST2477
1	Summary	The protein encoded by this gene is a plasma gly...
1	Orthologs	mouseall

7.medlineplus

category_crawl.py
Health Topics.txt
medlineplus_crawl.py

raw_data https://medlineplus.gov/healthtopics.html

category_crawl.py

input이 될 카테고리 수집

```python
url_requests = requests.get('https://medlineplus.gov/healthtopics.html')
soup = BeautifulSoup(url_requests.content,'html.parser')

alphabet_ul = soup.find('ul',{'class':'alpha-links'}).find_all('a')

alphabet_list = list()
f = open('Health Topics.txt','w')
for i in alphabet_ul:
    ALPHABET_URL = i['href']
    if ALPHABET_URL != 'https://medlineplus.gov/all_healthtopics.html':
        alphabet_list.append(ALPHABET_URL)

for url in alphabet_list:
    url_list = list()

    print(f'{alphabet_list.index(url)+1}/{len(alphabet_list)}')
    url_requests = requests.get(url)

    soup = BeautifulSoup(url_requests.content,'html.parser')

    section_ul = soup.find('div',{'id':'topic_byalpha'}).find('article').find('section').find_all('a')

    for li in section_ul:
        f.write(f"{li['href']}\n")
```

7.medlineplus

category_crawl.py
Health Topics.txt
medlineplus_crawl.py

raw_data https://medlineplus.gov/healthtopics.html

medlineplus_crawl.py

카테고리 리스트를
input으로하여 크롤링

```
"""
    모든 crawl_dict에 들어가는 변수 처리 부분
"""
topic_name = soup.find('h1',{'itemprop':'name'}).get_text().replace('\n','').replace('\t','').replace('  ','').
try:
    topic_name_also = soup.find('span',{'class':'alsocalled'}).get_text().replace('\n','').replace('\t','').rep
except:
    topic_name_also = None


"""
    main_div로 부터 section crawl 추출
"""
main_div = soup.find('div',{'class':'main'})

###summary 추출
summary_dict = dict()
summary_dict['url'] = category_url
summary_dict['topic_name'] = topic_name
summary_dict['topic_name_also'] = topic_name_also
summary_dict['title'] = 'Summary'
summary_div = main_div.find('div',{'id':'topic-summary'})
summary_dict['content'] = summary_div.get_text().replace('\n','').replace('\t','').replace('  ','').strip()
```

7.medlineplus (medlineplus)

url	topic_name	topic_name_also	title	content	link
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Summary	Your abdomen extends from below your chest to...	(NULL)
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Start Here	Abdominal Pain (Mayo Foundation for Medical Edu...	['https://www.mayoclinic.org/symptoms/abdomin...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Diagnosis and Tests	Abdominal and Pelvic CT (Computed Tomography...	['https://www.radiologyinfo.org/en/info/abdomin...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Specifics	Functional Dyspepsia (Mayo Foundation for Medic...	['https://www.mayoclinic.org/diseases-conditions...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Clinical Trials	ClinicalTrials.gov: Abdominal Pain (National Institut...	['https://clinicaltrials.gov/search/open/condition=...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Journal Articles Refere...	Article: Analysis of clinical characteristics of centrall...	['https://www.ncbi.nlm.nih.gov/pubmed/364825...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Find an Expert	American College of GastroenterologyNational Ins...	['https://gi.org', 'https://www.niddk.nih.gov']
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Children	Children's (Pediatric) Ultrasound - Abdomen (Ame...	['https://www.radiologyinfo.org/en/info/abdomus...
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Teenagers	Stomachaches (For Teens) (Nemours Foundation...	['https://kidshealth.org/en/teens/stomachaches....
https://medlineplus.gov/abdominalpain.html	Abdominal Pain	Bellyache	Patient Handouts	Abdominal pain (Medical Encyclopedia) Also in Spa...	['https://medlineplus.gov/ency/article/003120.ht...

8.한의학 (pharmdbk_tkm_relation & oasis_clinical & oasis_preclinical & tkm_ref_tb)

raw_data

medlineplus.sql 2023-01-1

유형: SQL-Script
크기: 7.65MB
수정한 날짜: 2023-01-11 오전 9:51

tkm_id	relation_type	tagged_relationship	tkm_name	kinds	pmid	data_source	data_source_relationship
M0000146	related Drug	TKM-Drug	Cassiae Semen	1,4,5-Trihydroxy-2-methoxy-7-methyl-9,10-anthra...		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	1-triacontanol		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	2,5-dimethoxy-4-benzoquinone		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	6-demethoxycapillarisin		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	9,10-Anthracenedione, 1-hydroxy-8-methoxy-3-m...		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	AGN-PC-03QP9L		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling
M0000146	related Drug	TKM-Drug	Cassiae Semen	AGN-PC-0505T5		Korean_Traditional_Knowledge_Portal_20140617	Chemical_Profiling

tkm_key	tkm_kor	tkm_chn	tkm_eng	category	clinical_target	patient_info	intake_info	effect	concurrent_infusion	ref_no
TKM001	가자	訶子	Terminaliae Fructus	소화 계통의 질병	담낭염, Calculus of bile...	- 치료군 30례- 대조군 2...	치료군은 조영 1-2시간 전에 가...	- 담낭조영 관찰이 시험군(80%)이 대...	-	166
TKM002	갈근	葛根	Puerariae Radix	순환 계통의 질병	급성뇌경색, Acute isch...	- 치료군 30례- 대조군 3...	치료군은 전탕액 100ml/회, 3...	- 두 그룹간 임상치료 효과에서 차이가...	-	44
TKM002	갈근	葛根	Puerariae Radix	순환 계통의 질병	부정맥, arrhythmia	-	-	-임상효과 있음	-	32

tkm_key	tkm_kor	tkm_chn	tkm_eng	category	preclinical_target	model_info_1	model_info_2	effect	ref_no
TKM001	가자	訶子	Terminaliae Fructus	다른 곳에서 분류되지 않...	항산화	In vivo	female albino mice	DMBA/Croton Oil-induced model - endogenous cy...	150
TKM001	가자	訶子	Terminaliae Fructus	다른 곳에서 분류되지 않...	항산화	In vitro	정보 없음	LOX 억제	42
TKM001	가자	訶子	Terminaliae Fructus	소화 계통의 질병	간섬유증예방	In vitro	LX-2 cell	ROS 생성 완화, ERK 인산화 유도, total Nrf2 발현 증가,...	87

tkm_key	tkm_kor	tkm_chn	tkm_eng
TKM001	가자	訶子	Terminaliae Fructus
TKM002	갈근	葛根	Puerariae Radix
TKM003	갈화	葛花	Puerariae Flos

9.의약품 데이터 (medicine)

raw_data

식약처 의약품안전나라 의약품통합정보... 2023-03-17 오전 9:46

식약처 의약품안전나라 의약품통합정보시스템_데이터_230113.xlsx
유형: Microsoft Excel 워크시트
만든 이: pc-2
크기: 9.80MB
수정한 날짜: 2023-03-17 오전 9:46

TKM_KEY	품목기준코드	제품명	제품영문명	업체명	허가일	품목구분	허가번호	취소/취하	취소/취하일자	주성분	첨가제	품목분류
TKM001	200604726	경동한방솔루션가자		(주)경동한방제약	2006-04-18	한약(생약)제제등	113	정상			가자	
TKM001	200005710	경방향성파적환엑스과립		경방신약(주)	2000-09-29	한약(생약)제제등	133	정상		감초/박하/가자/길경/사인/...	유당수화물,이상물건조엑스,...	
TKM001	200609207	경신가자		경신제약(주)	2006-06-28	한약(생약)제제등	118	폐업	2020-01-02	가자		
TKM001	200504842	경희한약가자		학교법인 경희학원(경희한약)	2005-06-03	한약(생약)제제등	150	정상		가자		

품목분류	전문의약품	완제/원료	허가/신고	제조/수입	마약구분	모양	색상	제형	장축	단축	신약구분	표준코드명	ATC코드	묶음의약품정보	e은약요	수입제조국	주성분영문
	한약재	한약재	신고	제조							N						
	일반의약품	완제의약품	신고	제조							N	8.81E+64	R02A				Licorice/Mentha Herb/Terminalia ...
	한약재	한약재	신고	제조							N						Terminalia Fruit
	한약재	한약재	신고	제조							N	8.80E+38					Terminalia Fruit

2. VCF > Final Table (parsing & join)

VCF parsing

input_data	parsing 후 테이블명
[사용자ID]-[등록일자]-[순번(001)].vcf	**[VCF]**

DB join

테이블	connect_key
[VCF]-dbSNP	**chrom_pos_ref_alt**
[VCF-dbSNP]-GWAS	**rsID_ALT**
[VCF-dbSNP-GWAS]-1KGP	**chrom_pos_ref_alt**
[VCF-dbSNP-GWAS-1KGP]-uniprot	**rsID** (vcf parsing때 생성된 allel1, allel2와 매칭되는 경우와 안되는 경우를 구분하여 join함)

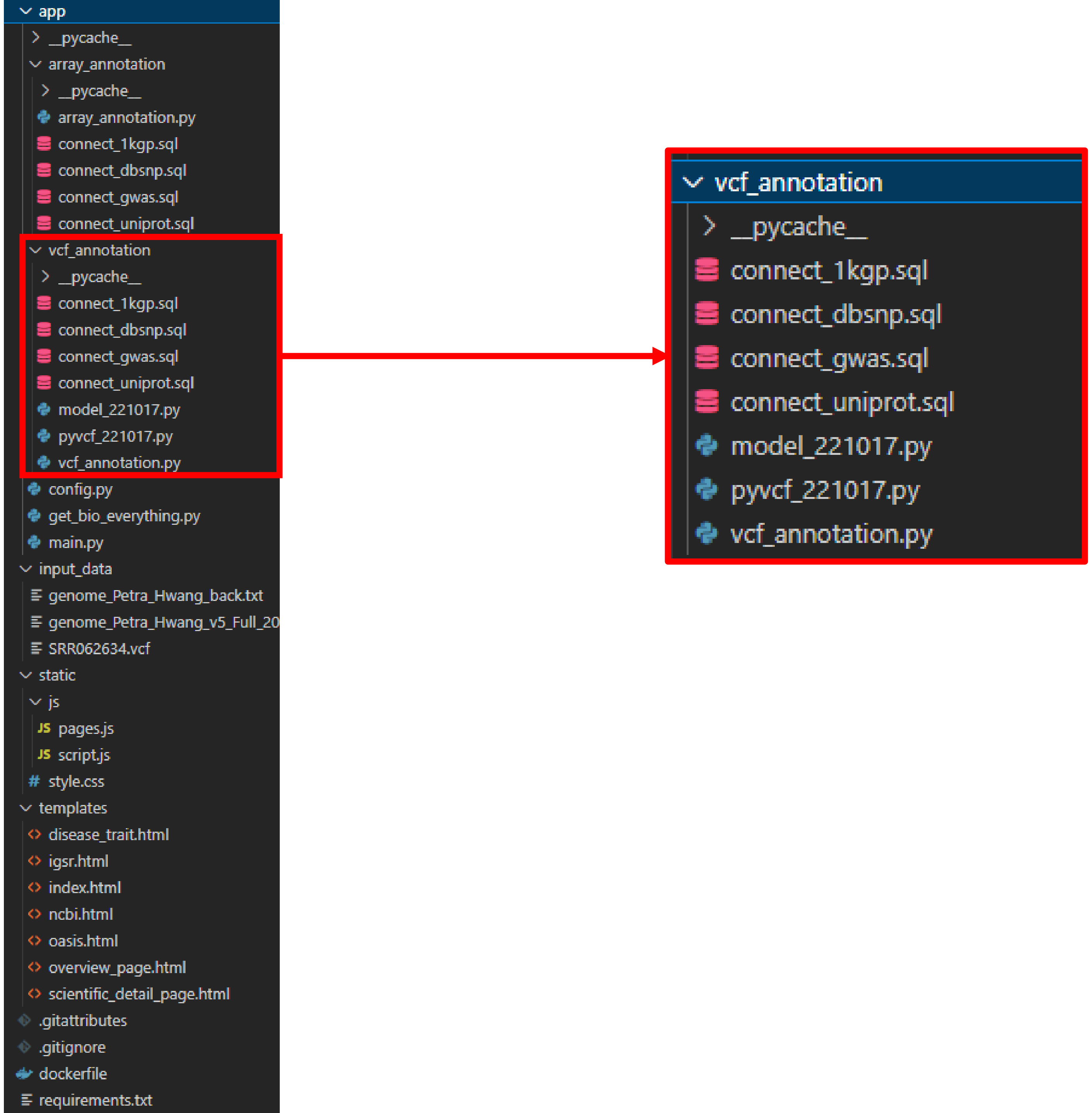
Final Table

[VCF-dbSNP-GWAS-1KGP-uniprot]	[사용자ID]-[등록일자]-[순번(001)]_VCF

VCF > vcf_{사람고유id}

I. 파이썬 소스코드 pyvcf.py를 활용하여 VCF 파일을 parsing한다.
II. pyvcf의 parsing정보를 활용하여 genome_type의 Allele_1, Allele_2 와 key 값 (chrom-pos-ref-alt)을 column 추가한다.
III. DB에 저장한다. (테이블 명 : vcf_{사람고유id})

2-1. VCF_annotation



2-1. VCF_annotation

raw_data

SRR062634.vcf
유형: vCalendar 파일
크기: 190MB
수정한 날짜: 2022-04-06 오전 11:25

chrom	pos	id	ref	alt	qual	filter	info	format	sample_data	allele_1	allele_2	connect_key
1	14,907	None	A	G	68.84	None	{'AC': [2], 'AF': [1.0], 'AN': 2, 'DP': 3, 'ExcessHet...	GT:AD:DP:GQ:PL	1/1;[0, 3];3;9;[82, 9, 0]	G	G	1-14907-A-G
1	14,930	None	A	G	53.32	None	{'AC': [2], 'AF': [1.0], 'AN': 2, 'DP': 2, 'ExcessHet...	GT:AD:DP:GQ:PL	1/1;[0, 2];2;6;[65, 6, 0]	G	G	1-14930-A-G
1	16,378	None	T	C	137.3	None	{'AC': [2], 'AF': [1.0], 'AN': 2, 'BaseQRankSum': ...	GT:AD:DP:GQ:PL	1/1;[1, 6];7;11;[151, 11, 0]	C	C	1-16378-T-C
1	16,497	None	A	G	88.64	None	{'AC': [1], 'AF': [0.5], 'AN': 2, 'BaseQRankSum': -...	GT:AD:DP:GQ:PL	0/1;[6, 5];11;96;[96, 0, 160]	A	G	1-16497-A-G
1	16,534	None	C	T	43.65	None	{'AC': [1], 'AF': [0.5], 'AN': 2, 'BaseQRankSum': ...	GT:AD:DP:GQ:PL	0/1;[1, 3];4;15;[51, 0, 15]	C	T	1-16534-C-T
1	16,571	None	G	A	43.65	None	{'AC': [1], 'AF': [0.5], 'AN': 2, 'BaseQRankSum': -...	GT:AD:DP:GQ:PL	0/1;[1, 3];4;15;[51, 0, 15]	G	A	1-16571-G-A
1	39,261	None	T	C	46.32	None	{'AC': [2], 'AF': [1.0], 'AN': 2, 'DP': 2, 'ExcessHet...	GT:AD:DP:GQ:PL	1/1;[0, 2];2;6;[58, 6, 0]	C	C	1-39261-T-C
1	54,490	None	G	A	69.84	None	{'AC': [2], 'AF': [1.0], 'AN': 2, 'DP': 3, 'ExcessHet...	GT:AD:DP:GQ:PL	1/1;[0, 3];3;9;[83, 9, 0]	A	A	1-54490-G-A

11	allele_1
12	allele_2
13	connect_key

dbsnp join하는 key

vcf의 데이터를 그대로 가져오지 않고 python에서 로직을 더하여
allele_1, allele_2, connect_key를 추가하였다.

allele_1, allele_2 : sample_data열의 ?/?구조와 ref , alt열의 값을 매칭하여 추가
connect_key : chrom, pos, ref, alt 열을 나열한 값으로 join을 위한 key값 추가

2-1. VCF_annotation

pyvcf 활용 vcf파일 parsing

```
vcf_reader = pyvcf.Reader(open(vcf_file, 'r'))
genom_length = vcf_counter(vcf_reader.reader)
vcf_reader = pyvcf.Reader(open(vcf_file, 'r'))
vcf_list = list()

for i in tqdm(range(genom_length)):
    row_vcf_reader = vcf_reader.next()
    vcf_dict = {'chrom' : str(row_vcf_reader.CHROM),
                'pos' : int(row_vcf_reader.POS),
                'id' : str(row_vcf_reader.ID),
                'ref' : str(row_vcf_reader.REF),
                'alt' : row_vcf_reader.ALT[0],
                'qual' : str(row_vcf_reader.QUAL),
                'filter' : str(row_vcf_reader.FILTER) if str(row_vcf_reader.FILTER) != "[]" else 'PASS',
                'info' : str(row_vcf_reader.INFO),
                'format': str(row_vcf_reader.FORMAT)}
    sample=row_vcf_reader.genotype("SRR062634")
    sample_data_txt = ''
```

내부 로직상 요구되는 열 추가

```
vcf_dict['sample_data'] = str(sample_data_txt)
vcf_dict['allele_1'] = gt_bases_sep(str(sample.gt_bases))[0]
vcf_dict['allele_2'] = gt_bases_sep(str(sample.gt_bases))[1]
vcf_dict['connect_key'] = f"{str(vcf_dict['chrom'])}-{str(vcf_dict['pos'])}-{str(vcf_dict['ref'])}-{str(vcf_dict['alt'])}"
vcf_list.append(vcf_dict)
```

DB적재

```
vcf_df = pd.DataFrame(vcf_list)
vcf_df.to_sql(name='vcf',con=db_connection, if_exists='append', index=False)

cursor.execute('ALTER TABLE `vcf` ADD INDEX `connect_key` (`connect_key`);')
mydb.commit()
```

1. **connect_dbsnp** > vcf_dbsnp_tb
    - key 값(chrom-pos-ref-alt)기준 쿼리문으로 연결하여 VCF에 rsID를 추가한다.
2. **connect_gwas** > vcf_dbsnp_gwas
    - key 값(rsID)기준 쿼리문으로 연결하여 질병관련정보를 추가한 뒤, Effect_allele 값과 Allele_1, Allele_2 값을 매칭하는 copy열을 추가한다.
    - risk_type 열 추가
    - risk_score 열 추가
3. **connect_1kgp** > vcf_dbsnp_gwas_1kgp
    - key 값(chrom-pos-ref-alt)기준 쿼리문으로 연결하여 인종분포도를 추가한다.
4. **connect_uniprot** > vcf_dbsnp_gwas_1kgp_uniport
    - key 값(rsID)기준 쿼리문으로 Join함
    - vcf parsing때 생성된 allel1, allel2와 매칭되는 경우와 안되는 경우를 구분하여 join함

1. connect_dbsnp > vcf_dbsnp_tb

connect_dbsnp.sql
[VCF]-dbSNP

```
create table __bio_db.connect_vcf_dbsnp
SELECT a.*, b.rsID_ALT, b.rsID FROM __bio_db.vcf AS a
JOIN __bio_db.dbsnp37_chr1 AS b
ON a.connect_key = b.connect_key
;

insert into __bio_db.connect_vcf_dbsnp
SELECT a.*, b.rsID_ALT, b.rsID FROM __bio_db.vcf AS a
JOIN __bio_db.dbsnp37_chr2 AS b
ON a.connect_key = b.connect_key
;
```

chrom-pos-ref-alt dbsnp를 key값으로 하여 dbsnp에서 제공하는 rsID 추가

chrom_id 별로 나눠서 Join

chrom	pos	id	ref	alt	qual	filter	info	format	sample_data	allele_1	allele_2	connect_key	rsID_ALT	rsID
1	14,907	None	A	G	68.84	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[0, 3];3;9;[82...	G	G	1-14907-A-G	rs6682375-G	rs6682375
1	14,930	None	A	G	53.32	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[0, 2];2;6;[65...	G	G	1-14930-A-G	rs6682385-G	rs6682385
1	16,378	None	T	C	137.3	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[1, 6];7;11;[1...	C	C	1-16378-T-C	rs148220436-C	rs148220436
1	16,497	None	A	G	88.64	None	{'AC': [1], 'AF': [0.5],...	GT:AD:DP:GQ:PL	0/1;[6, 5];11;96;[...	A	G	1-16497-A-G	rs150723783-G	rs150723783

1. connect_dbsnp > vcf_dbsnp_tb

#	이름
1	chrom
2	pos
3	id
4	ref
5	alt
6	qual
7	filter
8	info
9	format
10	sample_data
11	allele_1
12	allele_2
13	connect_key
14	rsID_ALT
15	rsID

dbsnp에서 제공한 값

2. connect_gwas > vcf_dbsnp_gwas

connect_gwas.sql
[VCF-dbSNP]-gwas

```sql
create table __bio_db.connect_vcf_dbsnp_gwas_temp
SELECT
    a.chrom,a.pos,a.id,a.ref,a.alt,a.qual,a.filter,a.info,a.`format`,a.sample_data,a.allele_1,a.allele_2,b.*,a.connect_key,
    CASE
        WHEN b.EFFECT_ALLELE = '?' THEN '?'
        WHEN b.EFFECT_ALLELE = a.allele_1 AND b.EFFECT_ALLELE = a.allele_2 THEN '2'
        WHEN b.EFFECT_ALLELE != a.allele_1 AND b.EFFECT_ALLELE = a.allele_2 THEN '1'
        WHEN b.EFFECT_ALLELE = a.allele_1 AND b.EFFECT_ALLELE != a.allele_2 THEN '1'
        ELSE '0'
    END AS copies,
    CASE
        WHEN b.OR = "" THEN ""
        WHEN b.CI LIKE '%increase' THEN 'BETA'
        WHEN b.CI LIKE '%decrease' THEN 'BETA'
        ELSE 'OR'
    END AS risk_type
FROM __bio_db.connect_vcf_dbsnp AS a
JOIN __bio_db.gwas_catelog AS b
ON a.rsID_ALT = b.STRONGEST_SNP
;
```

rsID_ALT를 key값으로 하여 gwas에서 제공하는 질병관련정보를 추가
Effect_allele 값과 Allele_1, Allele_2값을 매칭하는 copy열을 추가
raw data에서 내부로직에 따라 risk_type열을 생성

```sql
create table __bio_db.connect_vcf_dbsnp_gwas
SELECT
    a.*,
    CASE
        WHEN a.`OR` = '' THEN ''
        WHEN a.copies = '?' THEN '?'
        WHEN a.copies = '0' THEN '0%'
        WHEN a.risk_type = 'OR' THEN CONCAT(CAST((CAST(a.`OR` AS FLOAT) - 1) * 100 * CAST(a.copies AS FLOAT) AS signed),"%")
        WHEN a.risk_type = 'BETA' and a.CI LIKE '%increase' THEN CONCAT(CAST(CAST(a.`OR` AS FLOAT) * 100 * CAST(a.copies AS FLOAT) AS signed),"%")
        WHEN a.risk_type = 'BETA' and a.CI LIKE '%decrease' THEN CONCAT("-",CAST(CAST(a.`OR` AS FLOAT) * 100 * CAST(a.copies AS FLOAT) AS SIGNED),"%")
        ELSE ''
    END AS risk_score
FROM __bio_db.connect_vcf_dbsnp_gwas_temp AS a
;
```

내부 로직에 따라 risk_type과 copy열을 참조하여 risk_type열 추가

2. connect_gwas > vcf_dbsnp_gwas

#	이름
1	chrom
2	pos
3	id
4	ref
5	alt
6	qual
7	filter
8	info
9	format
10	sample_data
11	allele_1
12	allele_2
13	DATE_ADDED_TO_...
14	PUBMEDID
15	AUTHOR
16	DATE
17	JOURNAL
18	LINK
19	STUDY
20	DISEASE_TRAIT
21	INIT_SAMP_SIZE
22	REP_SAMP_SIZE
23	REGION
24	CHR_ID
25	CHR_POS
26	REPORTED_GENE
27	MAPPED_GENE
28	UPSTREAM_GENE_ID
29	DOWNSTREAM_GE...
30	SNP_GENE_IDS
31	UPSTREAM_DIST
32	DOWNSTREAM_DIST
33	STRONGEST_SNP
34	EFFECT_ALLELE
35	SNPS
36	MERGED
37	SNP_ID_CURRENT
38	var_type
39	INTERGENIC
40	RISK_AF
41	PVALUE
42	PVALUE_MLOG
43	PVALUE_TEXT
44	OR
45	CI
46	PLATFORM
47	CNV
48	connect_key
49	copies
50	risk_type
51	risk_score
52	risk_level

gwas에서 제공한 값

내부로직에 따라 추가한 값

3. connect_1kgp > vcf_dbsnp_gwas_1kgp

connect_1kgp.sql

[VCF-dbSNP-gwas]-1kgp

```sql
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp
with tb as(
    select * from connect_vcf_dbsnp_gwas
    where chrom = '1'
)
select tb.*, b.ACB, b.ASW, b.BEB, b.CDX, b.CEU, b.CHB, b.CHS, b.CLM, b.ESN, b.FIN, b.GBR, b.GIH, b.GWD, b.IBS,
from tb
left outer JOIN `1000genome_chr1` b
ON tb.connect_key = b.connect_key
;

insert into connect_vcf_dbsnp_gwas_1kgp
with tb as(
    select * from connect_vcf_dbsnp_gwas
    where chrom = '2'
)
select tb.*, b.ACB, b.ASW, b.BEB, b.CDX, b.CEU, b.CHB, b.CHS, b.CLM, b.ESN, b.FIN, b.GBR, b.GIH, b.GWD, b.IBS,
from tb
left outer JOIN `1000genome_chr2` b
ON tb.connect_key = b.connect_key
;
```

chrom-pos-ref-alt dbsnp를 key값으로 하여 1kgp에서 제공하는 인종분포도 추가

chrom_id 별로 나눠서 Join

3. connect_1kgp > vcf_dbsnp_gwas_1kgp

#	이름
1	chrom
2	pos
3	id
4	ref
5	alt
6	qual
7	filter
8	info
9	format
10	sample_data
11	allele_1
12	allele_2
13	DATE_ADDED_TO_...
14	PUBMEDID
15	AUTHOR
16	DATE
17	JOURNAL
18	LINK
19	STUDY
20	DISEASE_TRAIT
21	INIT_SAMP_SIZE
22	REP_SAMP_SIZE
23	REGION
24	CHR_ID
25	CHR_POS
26	REPORTED_GENE
27	MAPPED_GENE
28	UPSTREAM_GENE_ID
29	DOWNSTREAM_GE...
30	SNP_GENE_IDS
31	UPSTREAM_DIST
32	DOWNSTREAM_DIST
33	STRONGEST_SNP
34	EFFECT_ALLELE
35	SNPS
36	MERGED
37	SNP_ID_CURRENT
38	var_type
39	INTERGENIC
40	RISK_AF
41	PVALUE
42	PVALUE_MLOG
43	PVALUE_TEXT
44	OR
45	CI
46	PLATFORM
47	CNV
48	connect_key
49	copies
50	risk_type
51	risk_score
52	risk_level
53	ACB
54	ASW
55	BEB
56	CDX
57	CEU
58	CHB
59	CHS
60	CLM
61	ESN
62	FIN
63	GBR
64	GIH
65	GWD
66	IBS
67	ITU
68	JPT
69	KHV
70	LWK
71	MSL
72	MXL
73	PEL
74	PJL
75	PUR
76	STU
77	TSI
78	YRI

1kgp에서 제공한 값

4. connect_uniprot > vcf_dbsnp_gwas_1kgp_uniport

connect_uniprot.sql

[VCF-dbSNP-gwas-1kgp]-uniprot

```
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp_uniprot_temp
SELECT a.*, b.`GENENAME`, b.`VARIANT AA CHANGE`, b.`CONSEQUENCE TYPE`, b.`UNIPROT_REF`, b.`UNIPROT_ALT`
from connect_vcf_dbsnp_gwas_1kgp a
left outer JOIN `uniprot` b
ON a.SNPS = b.`SOURCE DB ID`
;
```

rsID를 key값으로 하여 uniprot에서 제공하는 유전자 정보 추가

```
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp_uniprot_matched
SELECT * FROM connect_vcf_dbsnp_gwas_1kgp_uniprot_temp a WHERE a.UNIPROT_ALT = a.allele_1 OR a.UNIPROT_ALT = a.allele_2
;
```

outer join 된 테이블에서 uniport의 allele값과 vcf에서 제공하는 allele값이 일치하는 경우만 분리하여 최종테이블에 create하여 적재

```
CREATE table connect_vcf_dbsnp_gwas_1kgp_uniprot
SELECT * FROM connect_vcf_dbsnp_gwas_1kgp_uniprot_temp a WHERE a.`VARIANT AA CHANGE` IS NULL
;

insert into connect_vcf_dbsnp_gwas_1kgp_uniprot
SELECT * from connect_vcf_dbsnp_gwas_1kgp_uniprot_matched
;
```

outer join 된 테이블에서 VARIANT AA CHANGE가 null값인 경우만 분리하여 최종테이블에 append하여 적재

4. connect_uniprot > vcf_dbsnp_gwas_1kgp_uniport

#	이름	#	이름	#	이름
1	CHR	26	EFFECT_ALLELE	51	CHS
2	pos	27	SNPS	52	CLM
3	allele_1	28	MERGED	53	ESN
4	allele_2	29	SNP_ID_CURRENT	54	FIN
5	DATE_ADDED_TO_...	30	var_type	55	GBR
6	PUBMEDID	31	INTERGENIC	56	GIH
7	AUTHOR	32	RISK_AF	57	GWD
8	DATE	33	PVALUE	58	IBS
9	JOURNAL	34	PVALUE_MLOG	59	ITU
10	LINK	35	PVALUE_TEXT	60	JPT
11	STUDY	36	OR	61	KHV
12	DISEASE_TRAIT	37	CI	62	LWK
13	INIT_SAMP_SIZE	38	PLATFORM	63	MSL
14	REP_SAMP_SIZE	39	CNV	64	MXL
15	REGION	40	connect_key	65	PEL
16	CHR_ID	41	copies	66	PJL
17	CHR_POS	42	risk_type	67	PUR
18	REPORTED_GENE	43	risk_score	68	STU
19	MAPPED_GENE	44	risk_level	69	TSI
20	UPSTREAM_GENE_ID	45	ACB	70	YRI
21	DOWNSTREAM_GE...	46	ASW	71	GENENAME
22	SNP_GENE_IDS	47	BEB	72	VARIANT AA CHAN...
23	UPSTREAM_DIST	48	CDX	73	CONSEQUENCE TYPE
24	DOWNSTREAM_DIST	49	CEU	74	UNIPROT_REF
25	STRONGEST_SNP	50	CHB	75	UNIPROT_ALT

uniprot에서 제공한 값

```
vcf : 272.15203 sec
connect_dbsnp
100%|
connect_dbsnp : 1873.62102 sec
connect_gwas
100%|
connect_gwas : 15.07489 sec
connect_1kgp
100%|
connect_1kgp : 2.83138 sec
connect_uniprot
100%|
connect_uniprot : 14.96050 sec
total time : 2178.70637 sec
```

vcf : 4m 32s

dbsnp : 31m 13s

gwas : 15s

1kgp : 3s

uniprot : 15s

———————————————————————————

total time : 36m 18s

3. Array Data Format > Final Table (parsing & join)

Array Data Format parsing

input_data	parsing 후 테이블명
[사용자ID]-[등록일자]-[순번(001)].txt	**[ArrayDataFormat]**

DB Join

테이블	connect_key
[ArrayDataFormat]-dbSNP	**chrom_pos_ref_alt**
[ArrayDataFormat-dbSNP]-GWAS	**rsID_ALT**
[ArrayDataFormat-dbSNP-GWAS]-1KGP	**chrom_pos_ref_alt**
[ArrayDataFormat-dbSNP-GWAS-1KGP]-uniprot	**rsID** (vcf parsing때 생성된 allel1, allel2와 매칭되는 경우와 안되는 경우를 구분하여 join함)

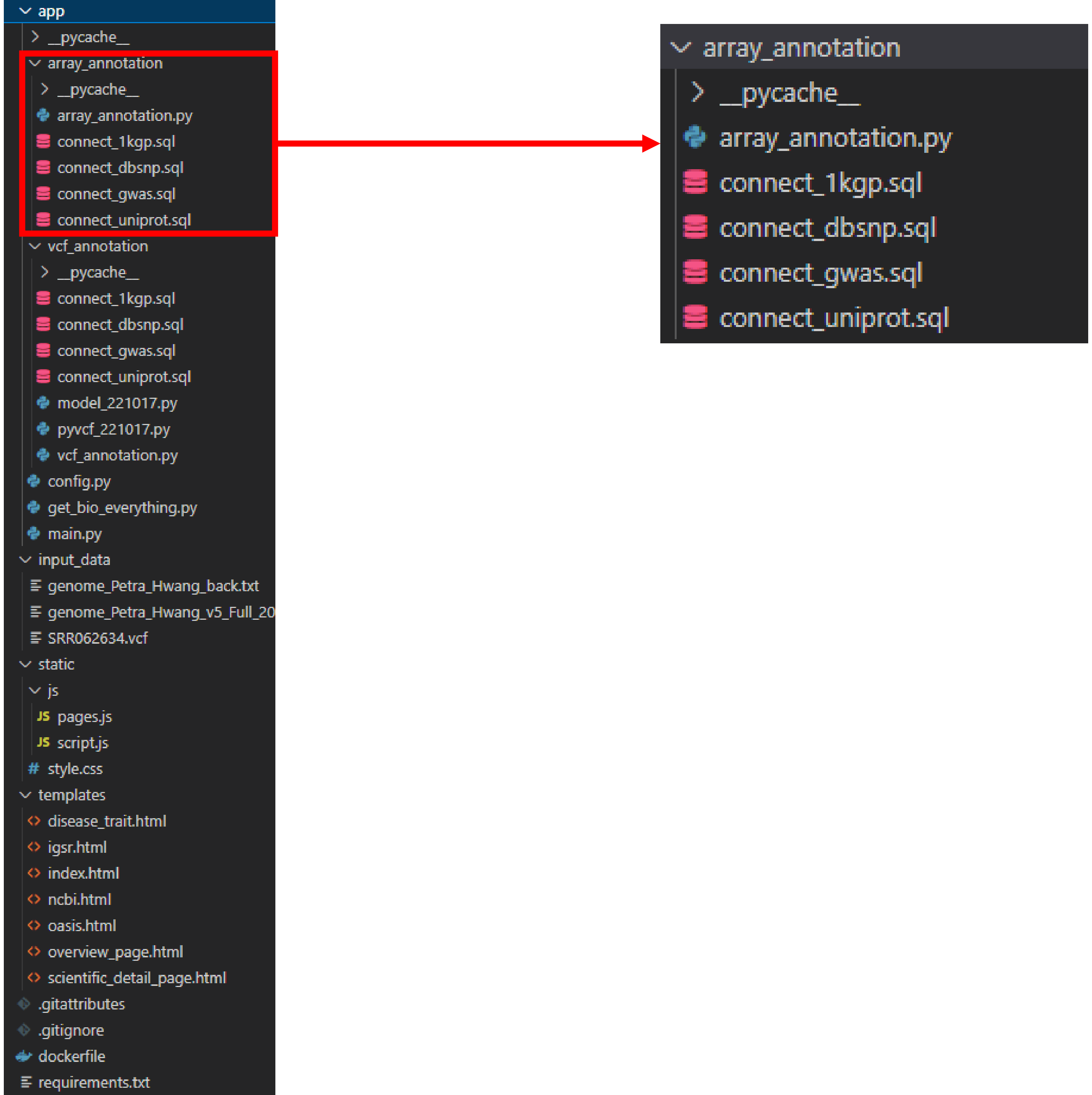
Final Table

[ArrayDataFormat-dbSNP-GWAS-1KGP-uniprot]	[사용자ID]-[등록일자]-[순번(001)]_ArrayDataFormat

Array Data Format > vcf_{사람고유id}

I. 리눅스 sed 혹은 파이썬 re 를 통해 raw 데이터에서 필요없는 라인을 삭제
II. pandas를 활용하여 genome_type의 Allele_1, Allele_2 와 key 값 (chrom-pos-ref-alt)을 column 추가한다.
III. DB에 저장한다. (테이블 명 : vcf_{사람고유id})

2-2. array_annotation



2-2. array_annotation

파이썬 내부 라이브러리 're'를 사용하여 리눅스 'sed'와 같은 역할 수행

```
with open(array_file, "r") as file:
    content = file.read()

new_content = re.sub(r'^#.*\n', '', content, flags=re.MULTILINE)

with open(array_file, "w") as file:
    file.write(new_content)
```

내부 로직상 요구되는 열 추가

```
vcf_df = pd.read_csv(array_file, sep='\t', names=['rsID','CHR','POS','genotype'])

geno = vcf_df.loc[:,['genotype']]
allele_list = list()
for i in tqdm(geno.values):
    allele_list.append(list(i[0]))

allele_df = pd.DataFrame(allele_list, columns=['allele_1','allele_2'])

final_df = pd.concat([vcf_df,allele_df],axis=1)
final_df = final_df.loc[:,['CHR','POS','rsID','allele_1','allele_2']]
```

DB적재

```
final_df.to_sql(name='vcf_petra',con=db_connection, if_exists='append', index=False)
cursor.execute('ALTER TABLE `vcf_petra` ADD INDEX `rsID` (`rsID`);')
mydb.commit()
```

2-2. array_annotation

raw_data

genome_Petra_Hwang_v5_Full_2022120...

genome_Petra_Hwang_v5_Full_20221205170
유형: 텍스트 문서
크기: 16.1MB

CHR	POS	rsID	allele_1	allele_2
1	69,869	rs548049170	T	T
1	567,092	rs9326622	C	C
1	727,841	rs116587930	G	G
1	752,721	rs3131972	G	G
1	754,105	rs12184325	C	C

array file의 데이터를 수정없이 그대로 가져옴

1. **connect_dbsnp** > vcf_dbsnp_tb
   - key 값(rsID)기준 쿼리문으로 연결하여 VCF에 chrom, pos, ref, alt를 추가한다.
2. **connect_gwas** > vcf_dbsnp_gwas
   - key 값(rsID)기준 쿼리문으로 연결하여 질병관련정보를 추가한 뒤, Effect_allele 값과 Allele_1, Allele_2 값을 매칭하는 copy열을 추가한다.
   - risk_type 열 추가
   - risk_score 열 추가
3. **connect_1kgp** > vcf_dbsnp_gwas_1kgp
   - key 값(chrom-pos-ref-alt)기준 쿼리문으로 연결하여 인종분포도를 추가한다.
4. **connect_uniprot** > vcf_dbsnp_gwas_1kgp_uniport
   - key 값(rsID)기준 쿼리문으로 Join함
   - vcf parsing때 생성된 allel1, allel2와 매칭되는 경우와 안되는 경우를 구분하여 join함

1. connect_dbsnp > vcf_dbsnp_tb

connect_dbsnp.sql
[VCF]-dbSNP

```
create table __bio_db.connect_vcf_dbsnp
SELECT a.*, b.rsID_ALT, b.rsID FROM __bio_db.vcf AS a
JOIN __bio_db.dbsnp37_chr1 AS b
ON a.connect_key = b.connect_key
;

insert into __bio_db.connect_vcf_dbsnp
SELECT a.*, b.rsID_ALT, b.rsID FROM __bio_db.vcf AS a
JOIN __bio_db.dbsnp37_chr2 AS b
ON a.connect_key = b.connect_key
;
```

chrom-pos-ref-alt dbsnp를 key값으로 하여 dbsnp에서 제공하는 rsID 추가

chrom_id 별로 나눠서 Join

chrom	pos	id	ref	alt	qual	filter	info	format	sample_data	allele_1	allele_2	connect_key	rsID_ALT	rsID
1	14,907	None	A	G	68.84	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[0, 3];3;9;[82...	G	G	1-14907-A-G	rs6682375-G	rs6682375
1	14,930	None	A	G	53.32	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[0, 2];2;6;[65...	G	G	1-14930-A-G	rs6682385-G	rs6682385
1	16,378	None	T	C	137.3	None	{'AC': [2], 'AF': [1.0],...	GT:AD:DP:GQ:PL	1/1;[1, 6];7;11;[1...	C	C	1-16378-T-C	rs148220436-C	rs148220436
1	16,497	None	A	G	88.64	None	{'AC': [1], 'AF': [0.5],...	GT:AD:DP:GQ:PL	0/1;[6, 5];11;96;[...	A	G	1-16497-A-G	rs150723783-G	rs150723783

1. connect_dbsnp > vcf_dbsnp_tb

#	이름
1	chrom
2	pos
3	id
4	ref
5	alt
6	qual
7	filter
8	info
9	format
10	sample_data
11	allele_1
12	allele_2
13	connect_key
14	rsID_ALT
15	rsID

dbsnp에서 제공한 값

2. connect_gwas > vcf_dbsnp_gwas

connect_gwas.sql

[VCF-dbSNP]-gwas

```sql
create table __bio_db.connect_vcf_dbsnp_gwas_temp
SELECT
    a.chrom,a.pos,a.id,a.ref,a.alt,a.qual,a.filter,a.info,a.`format`,a.sample_data,a.allele_1,a.allele_2,b.*,a.connect_key,
    CASE
        WHEN b.EFFECT_ALLELE = '?' THEN '?'
        WHEN b.EFFECT_ALLELE = a.allele_1 AND b.EFFECT_ALLELE = a.allele_2 THEN '2'
        WHEN b.EFFECT_ALLELE != a.allele_1 AND b.EFFECT_ALLELE = a.allele_2 THEN '1'
        WHEN b.EFFECT_ALLELE = a.allele_1 AND b.EFFECT_ALLELE != a.allele_2 THEN '1'
        ELSE '0'
    END AS copies,
    CASE
        WHEN b.OR = "" THEN ""
        WHEN b.CI LIKE '%increase' THEN 'BETA'
        WHEN b.CI LIKE '%decrease' THEN 'BETA'
        ELSE 'OR'
    END AS risk_type
FROM __bio_db.connect_vcf_dbsnp AS a
JOIN __bio_db.gwas_catelog AS b
ON a.rsID_ALT = b.STRONGEST_SNP
;
```

rsID_ALT를 key값으로 하여 gwas에서 제공하는 질병관련정보를 추가
Effect_allele 값과 Allele_1, Allele_2값을 매칭하는 copy열을 추가
raw data에서 내부로직에 따라 risk_type열을 생성

```sql
create table __bio_db.connect_vcf_dbsnp_gwas
SELECT
    a.*,
    CASE
        WHEN a.`OR` = '' THEN ''
        WHEN a.copies = '?' THEN '?'
        WHEN a.copies = '0' THEN '0%'
        WHEN a.risk_type = 'OR' THEN CONCAT(CAST((CAST(a.`OR` AS FLOAT) - 1) * 100 * CAST(a.copies AS FLOAT) AS signed),"%")
        WHEN a.risk_type = 'BETA' and a.CI LIKE '%increase' THEN CONCAT(CAST(CAST(a.`OR` AS FLOAT) * 100 * CAST(a.copies AS FLOAT) AS signed),"%")
        WHEN a.risk_type = 'BETA' and a.CI LIKE '%decrease' THEN CONCAT("-",CAST(CAST(a.`OR` AS FLOAT) * 100 * CAST(a.copies AS FLOAT) AS SIGNED),"%")
        ELSE ''
    END AS risk_score
FROM __bio_db.connect_vcf_dbsnp_gwas_temp AS a
;
```

내부 로직에 따라 risk_type과 copy열을 참조하여 risk_type열 추가

2. connect_gwas > vcf_dbsnp_gwas

#	이름
1	chrom
2	pos
3	id
4	ref
5	alt
6	qual
7	filter
8	info
9	format
10	sample_data
11	allele_1
12	allele_2
13	DATE_ADDED_TO_...
14	PUBMEDID
15	AUTHOR
16	DATE
17	JOURNAL
18	LINK
19	STUDY
20	DISEASE_TRAIT
21	INIT_SAMP_SIZE
22	REP_SAMP_SIZE
23	REGION
24	CHR_ID
25	CHR_POS
26	REPORTED_GENE
27	MAPPED_GENE
28	UPSTREAM_GENE_ID
29	DOWNSTREAM_GE...
30	SNP_GENE_IDS
31	UPSTREAM_DIST
32	DOWNSTREAM_DIST
33	STRONGEST_SNP
34	EFFECT_ALLELE
35	SNPS
36	MERGED
37	SNP_ID_CURRENT
38	var_type
39	INTERGENIC
40	RISK_AF
41	PVALUE
42	PVALUE_MLOG
43	PVALUE_TEXT
44	OR
45	CI
46	PLATFORM
47	CNV
48	connect_key
49	copies
50	risk_type
51	risk_score
52	risk_level

gwas에서 제공한 값

내부로직에 따라 추가한 값

3. connect_1kgp > vcf_dbsnp_gwas_1kgp

connect_1kgp.sql

[VCF-dbSNP-gwas]-1kgp

```sql
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp
with tb as(
    select * from connect_vcf_dbsnp_gwas
    where chrom = '1'
)
select tb.*, b.ACB, b.ASW, b.BEB, b.CDX, b.CEU, b.CHB, b.CHS, b.CLM, b.ESN, b.FIN, b.GBR, b.GIH, b.GWD, b.IBS,
from tb
left outer JOIN `1000genome_chr1` b
ON tb.connect_key = b.connect_key
;

insert into connect_vcf_dbsnp_gwas_1kgp
with tb as(
    select * from connect_vcf_dbsnp_gwas
    where chrom = '2'
)
select tb.*, b.ACB, b.ASW, b.BEB, b.CDX, b.CEU, b.CHB, b.CHS, b.CLM, b.ESN, b.FIN, b.GBR, b.GIH, b.GWD, b.IBS,
from tb
left outer JOIN `1000genome_chr2` b
ON tb.connect_key = b.connect_key
;
```

chrom-pos-ref-alt dbsnp를 key값으로 하여 1kgp에서 제공하는 인종분포도 추가

chrom_id 별로 나눠서 Join

3. connect_1kgp > vcf_dbsnp_gwas_1kgp

#	이름	#	이름	#	이름
1	chrom	27	MAPPED_GENE	53	ACB
2	pos	28	UPSTREAM_GENE_ID	54	ASW
3	id	29	DOWNSTREAM_GE...	55	BEB
4	ref	30	SNP_GENE_IDS	56	CDX
5	alt	31	UPSTREAM_DIST	57	CEU
6	qual	32	DOWNSTREAM_DIST	58	CHB
7	filter	33	STRONGEST_SNP	59	CHS
8	info	34	EFFECT_ALLELE	60	CLM
9	format	35	SNPS	61	ESN
10	sample_data	36	MERGED	62	FIN
11	allele_1	37	SNP_ID_CURRENT	63	GBR
12	allele_2	38	var_type	64	GIH
13	DATE_ADDED_TO_...	39	INTERGENIC	65	GWD
14	PUBMEDID	40	RISK_AF	66	IBS
15	AUTHOR	41	PVALUE	67	ITU
16	DATE	42	PVALUE_MLOG	68	JPT
17	JOURNAL	43	PVALUE_TEXT	69	KHV
18	LINK	44	OR	70	LWK
19	STUDY	45	CI	71	MSL
20	DISEASE_TRAIT	46	PLATFORM	72	MXL
21	INIT_SAMP_SIZE	47	CNV	73	PEL
22	REP_SAMP_SIZE	48	connect_key	74	PJL
23	REGION	49	copies	75	PUR
24	CHR_ID	50	risk_type	76	STU
25	CHR_POS	51	risk_score	77	TSI
26	REPORTED_GENE	52	risk_level	78	YRI

1kgp에서 제공한 값

4. connect_uniprot > vcf_dbsnp_gwas_1kgp_uniport

connect_uniprot.sql
[VCF-dbSNP-gwas-1kgp]-uniprot

```
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp_uniprot_temp
SELECT a.*, b.`GENENAME`, b.`VARIANT AA CHANGE`, b.`CONSEQUENCE TYPE`, b.`UNIPROT_REF`, b.`UNIPROT_ALT`
from connect_vcf_dbsnp_gwas_1kgp a
left outer JOIN `uniprot` b
ON a.SNPS = b.`SOURCE DB ID`
;
```

rsID를 key값으로 하여 uniprot에서 제공하는 유전자 정보 추가

```
CREATE TABLE connect_vcf_dbsnp_gwas_1kgp_uniprot_matched
SELECT * FROM connect_vcf_dbsnp_gwas_1kgp_uniprot_temp a WHERE a.UNIPROT_ALT = a.allele_1 OR a.UNIPROT_ALT = a.allele_2
;
```

outer join 된 테이블에서 uniport의 allele값과 vcf에서 제공하는 allele값이 일치하는 경우만 분리하여 최종테이블에 create하여 적재

```
CREATE table connect_vcf_dbsnp_gwas_1kgp_uniprot
SELECT * FROM connect_vcf_dbsnp_gwas_1kgp_uniprot_temp a WHERE a.`VARIANT AA CHANGE` IS NULL
;

insert into connect_vcf_dbsnp_gwas_1kgp_uniprot
SELECT * from connect_vcf_dbsnp_gwas_1kgp_uniprot_matched
;
```

outer join 된 테이블에서 VARIANT AA CHANGE가 null값인 경우만 분리하여 최종테이블에 append하여 적재

4. connect_uniprot > vcf_dbsnp_gwas_1kgp_uniport

#	이름	#	이름	#	이름
1	CHR	26	EFFECT_ALLELE	51	CHS
2	pos	27	SNPS	52	CLM
3	allele_1	28	MERGED	53	ESN
4	allele_2	29	SNP_ID_CURRENT	54	FIN
5	DATE_ADDED_TO_...	30	var_type	55	GBR
6	PUBMEDID	31	INTERGENIC	56	GIH
7	AUTHOR	32	RISK_AF	57	GWD
8	DATE	33	PVALUE	58	IBS
9	JOURNAL	34	PVALUE_MLOG	59	ITU
10	LINK	35	PVALUE_TEXT	60	JPT
11	STUDY	36	OR	61	KHV
12	DISEASE_TRAIT	37	CI	62	LWK
13	INIT_SAMP_SIZE	38	PLATFORM	63	MSL
14	REP_SAMP_SIZE	39	CNV	64	MXL
15	REGION	40	connect_key	65	PEL
16	CHR_ID	41	copies	66	PJL
17	CHR_POS	42	risk_type	67	PUR
18	REPORTED_GENE	43	risk_score	68	STU
19	MAPPED_GENE	44	risk_level	69	TSI
20	UPSTREAM_GENE_ID	45	ACB	70	YRI
21	DOWNSTREAM_GE...	46	ASW	71	GENENAME
22	SNP_GENE_IDS	47	BEB	72	VARIANT AA CHAN...
23	UPSTREAM_DIST	48	CDX	73	CONSEQUENCE TYPE
24	DOWNSTREAM_DIST	49	CEU	74	UNIPROT_REF
25	STRONGEST_SNP	50	CHB	75	UNIPROT_ALT

uniprot에서 제공한 값

```
vcf : 272.15203 sec
connect_dbsnp
100%|
connect_dbsnp : 1873.62102 sec
connect_gwas
100%|
connect_gwas : 15.07489 sec
connect_1kgp
100%|
connect_1kgp : 2.83138 sec
connect_uniprot
100%|
connect_uniprot : 14.96050 sec
total time : 2178.70637 sec
```

vcf : 4m 32s

dbsnp : 31m 13s

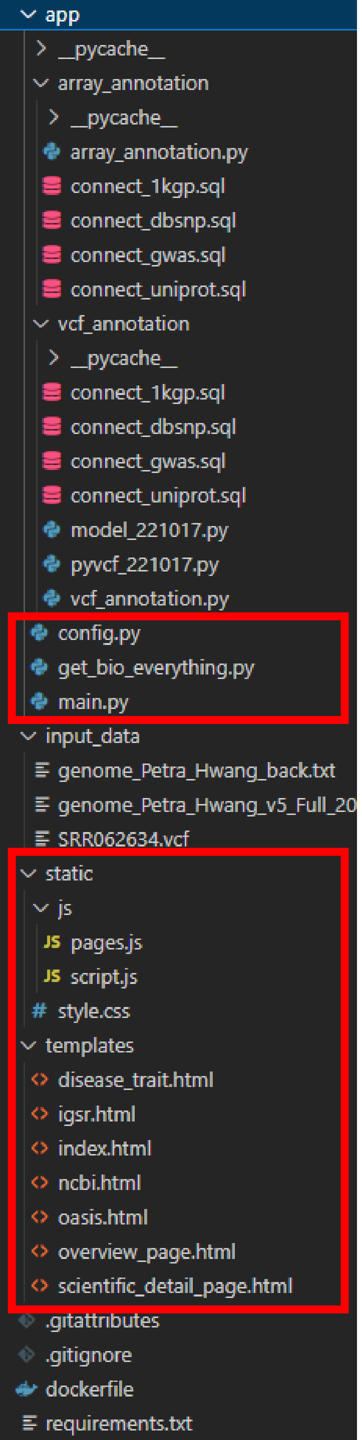
gwas : 15s

1kgp : 3s

uniprot : 15s

————————————————————————

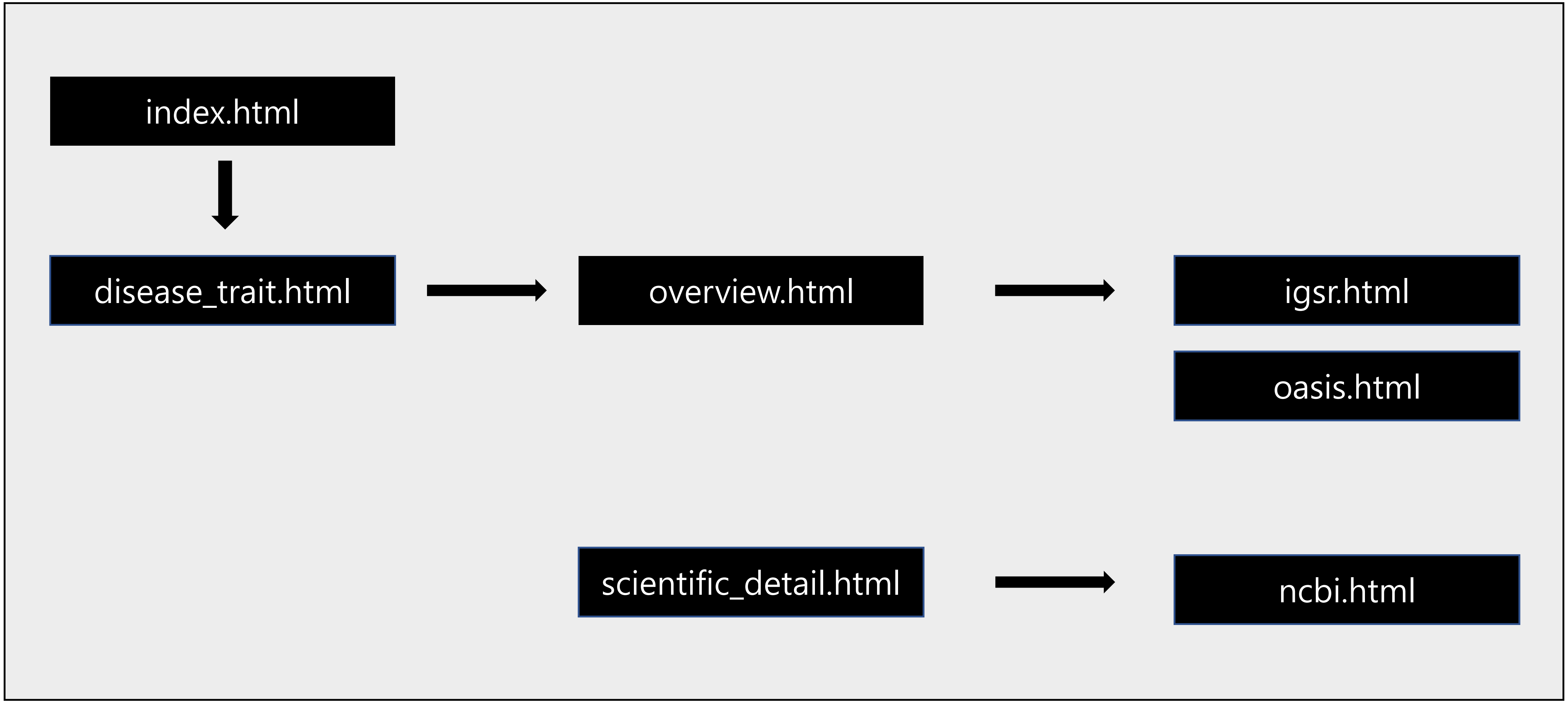
total time : 36m 18s



4. 웹화면 (FastAPI)

FastAPI

웹화면	역할
index.html	test 화면 (실시간 vcf_annotation기능)
disease_trait.html	사용자의 disease_trait 리스트를 뿌려줌 (현재 index.html 역할)
overview.html	disease_trait를 input으로 하는 메인 페이지
scientific_detail.html	disease_trait를 input으로 하는 디테일 페이지
igsr.html	인종별 분포도
oasis.html	tkm에 대한 임상, 전임상, 참고문헌 자료
ncbi.html	아미노산 변이에 대한 설명


index.html
disease_trait.html
overview.html
igsr.html
oasis.html
scientific_detail.html
ncbi.html

index.html

vcf 업로드 기능
array 파일 업로드 기능

User List에서 클릭시 해당 사용자의 최종 테이블을 참조한 화면 출력

index.html

vcf 업로드 기능

```
const upload_vcf_btn = document.querySelector("#upload_vcf_btn")
const upload_vcf_file = document.querySelector("#upload_vcf_file")
function click_upload_vcf_btn(e){
    e.preventDefault()

    var formdata = new FormData();
    formdata.append('file', upload_vcf_file.files[0])

    fetch('/uploadfile/',{
        method:'POST',
        body:formdata,
    })
    .then((response) => console.log(response.json()))
    .then(data => console.log(data))
};
upload_vcf_btn.addEventListener("click",click_upload_vcf_btn)
```

array file 업로드 기능

```
const upload_array_btn = document.querySelector("#upload_array_btn")
const upload_array_file = document.querySelector("#upload_array_file")
function click_upload_array_btn(e){
    e.preventDefault()

    var formdata = new FormData();
    formdata.append('file', upload_array_file.files[0])

    fetch('/uploadfile/array/',{
        method:'POST',
        body:formdata,
    })
    .then((response) => console.log(response.json()))
    .then(data => console.log(data))
};
upload_array_btn.addEventListener("click",click_upload_array_btn)
```

index.html (내부 JS)

```python
@app.post('/uploadfile/')
async def uploadfile(request: Request, file: UploadFile):
    UPLOAD_DIRECTORY = "./input_data"
    contents = await file.read()
    with open(os.path.join(UPLOAD_DIRECTORY, file.filename), "wb") as f:
        f.write(contents)
    annotation.vcf_annotation("./input_data/"+file.filename)
    return "uploaded vcf file"

@app.post('/uploadfile/array/')
async def uploadfile(request: Request, file: UploadFile):
    UPLOAD_DIRECTORY = "./input_data"
    contents = await file.read()
    with open(os.path.join(UPLOAD_DIRECTORY, file.filename), "wb") as f:
        f.write(contents)
    annotation.array_annotation("./input_data/"+file.filename)
    return "uploaded array file"

# 메인페이지
@app.get("/")
async def root(request:Request):
    return templates.TemplateResponse("index.html", {"request":request})
```

main.py

disease_trait.html

1명의 사람에 대한 질병 리스트

DISEASE_TRAIT
Bone mineral density (hip)
Lipoprotein-associated phospholipase A2 activity and mass
C-reactive protein
Blood pressure
Endometriosis
Lentiform nucleus volume
Life threatening arrhythmia
IgE grass sensitization
Airflow obstruction
Axial length
Multiple sclerosis
Smoking behavior
Crohn's disease
IgG glycosylation
Basal cell carcinoma

First Prev 1 2 3 4 5 Next Last

disease_trait.html

tabulator활용하여 테이블로 화면 출력

```
var disease_trait = {{disease_trait|tojson}};
var sch = location.search
var params = new URLSearchParams(sch);

var disease_trait_table = new Tabulator('#disease_trait',{
    data:disease_trait,
    pagination:"local",
    paginationSize:15,
    columns:[
        // {formatter:"rowSelection", titleFormatter:"rowSelection", hozAlign:"center",
        // headerSort:false
        // },
        {title:"DISEASE_TRAIT", field:"DISEASE_TRAIT", headerFilter:"input"},
    ],
});
disease_trait_table.on("rowClick",function(e,row){
    window.location.href = '/overview_page?bio_input='+row._row.data.DISEASE_TRAIT+'&&user='+params.get('user');
});
```

disease_trait.html (내부 JS)

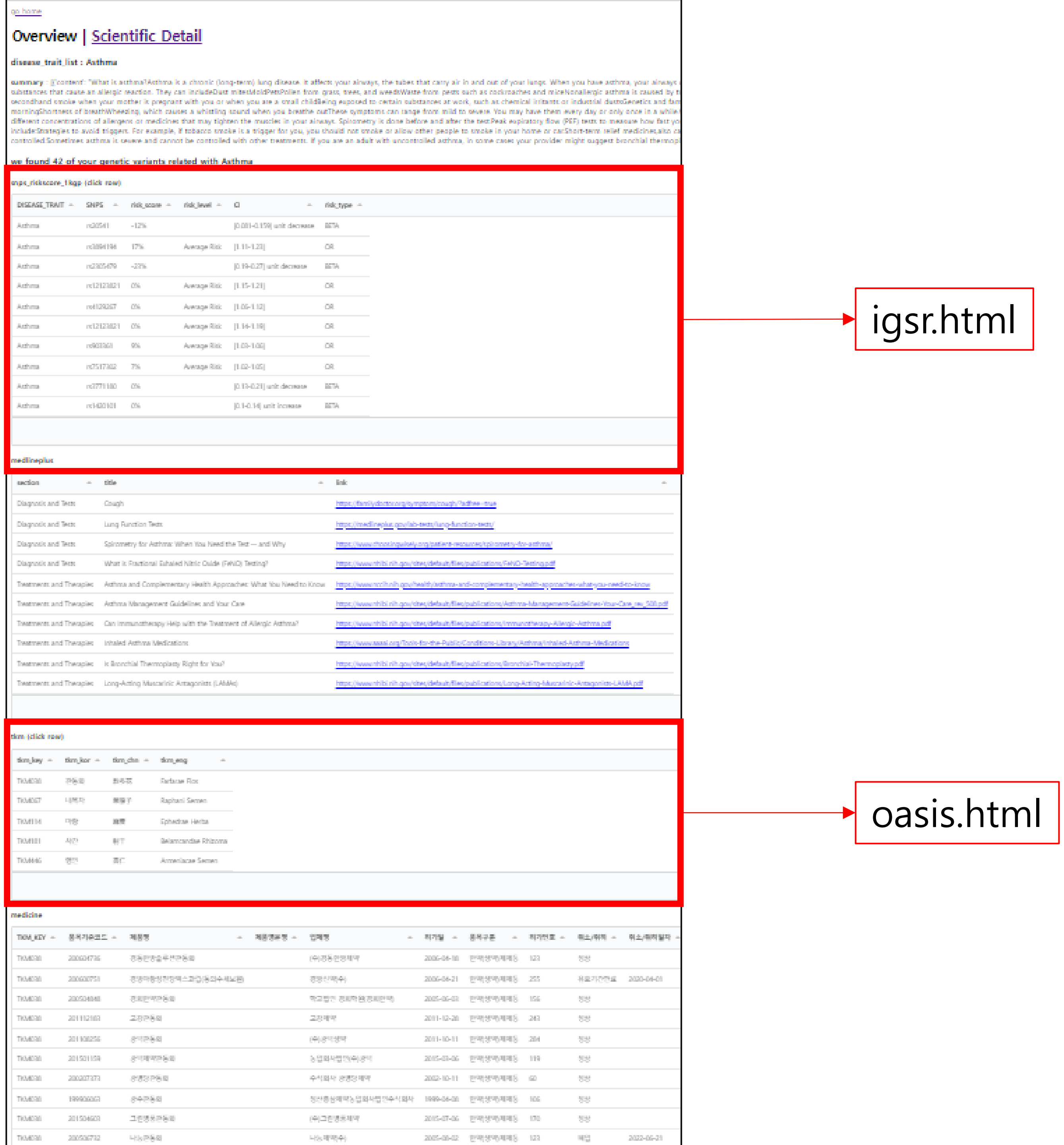
```
@app.get("/disease_trait")
# @app.get("/")
async def root(request:Request, user:str):
    disease_trait = get_bio_everything.get_disease_trait(user)
    return templates.TemplateResponse("disease_trait.html", {"request":request, "disease_trait":disease_trait, "user":user})
```

main.py

overview.html

질병명을 input으로 하여
medline의 summary와
annotation_table의 snps, riskscore,
1kgp의 정보 조회

tkm을 input으로 하여
medicine의 정보 조회



overview.html

tabulator활용하여 테이블로 화면 출력

```
var medlineplus = {{medlineplus|tojson}};
var medlineplus_table = new Tabulator('#medlineplus',{
    data:medlineplus,
    pagination:"local",
    paginationSize:10,
    columns:[
        {title:"section", field:"title"},
        {title:"title", field:"link_title"},
        {title:"link", field:"link", formatter:"link", formatterParams:{target:"_blank"}},
    ],
});
```

overview.html (내부JS)

```
@app.get("/overview_page")
async def root(request:Request, bio_input:str):
    summary = get_bio_everything.get_summary(bio_input)
    snps_riskscore_1kgp = get_bio_everything.get_snps_riskscore_1kgp(bio_input)
    snps_cnt = get_bio_everything.get_count_snps(bio_input)
    medlineplus = get_bio_everything.get_medlineplus(bio_input)
    tkm = get_bio_everything.get_tkm(bio_input)
    medicine = get_bio_everything.get_medicine(tkm)
    return templates.TemplateResponse("overview_page.html", {"request":request,
```

main.py

igsr.html

rsID를 input으로 하여
최종테이블의 인종별 분포도 조회

international genome sample resource

population	ratio [%]	superpopulation	pop_name
ACB	80	AFR	African Caribbean
ASW	81	AFR	African Ancestry SW
BEB	74	SAS	Bengali
CDX	59	EAS	Dai Chinese
CEU	77	EUR	CEPH
CHB	67	EAS	Han Chinese
CHS	60	EAS	Southern Han Chinese
CLM	69	AMR	Colombian
ESN	79	AFR	Esan
FIN	66	EUR	Finnish
GBR	84	EUR	British
GIH	79	SAS	Gujarati
GWD	80	AFR	Gambian Mandinka

First Prev 1 2 Next Last

tabulator활용하여 테이블로 화면 출력

```
var igsr = {{igsr|tojson}};
console.log(igsr)
var igsr_table = new Tabulator('#igsr',{
    data:igsr,
    pagination:"local",
    paginationSize:13,
    columns:[
        {title:"population", field:"key"},
        {title:"ratio [%]", field:"value"},
        {title:"superpopulation", field:"superpopulation"},
        {title:"pop_name", field:"pop_name"},
    ],
});
```

igsr.html (내부 JS)

```
@app.get("/igsr")
async def igsr(request:Request, snps:str, user:str):
    igsr = get_bio_everything.get_igsr(snps,user)
    return templates.TemplateResponse("igsr.html", {"request":request, "igsr":igsr})
```

main.py

oasis.html

질병명을 input으로 하여
oasis의 임상, 전임상 정보 조회
reference의 정보 조회

oasis_clinical

tkm_key	tkm_kor	tkm_chn	tkm_eng	category	clinical_target	patient_info	intake_info	effect	concurrent_infusion	ref_no
TKM038	관동화	款冬花	Farfarae Flos	-	-	-	-	-	-	0

oasis_preclinical

tkm_key	tkm_kor	tkm_chn	tkm_eng	category	preclinical_target	model_info_1	model_info_2	effect	ref_no
TKM038	관동화	款冬花	Farfarae Flos	내분비선 영양 및 순환계	항결핵	In vitro	Mycobacterium tuberculosis	항 결핵 활성.	22

oasis_reference

tkm_key	약재명	No	참고문헌	url
TKM038	관동화, 款冬花, Farfarae Flos	1	대한민국약전 제11개정	
TKM038	관동화, 款冬花, Farfarae Flos	2	조선민주주의인민공화국약전 제7판	
TKM038	관동화, 款冬花, Farfarae Flos	3	대만중약전	
TKM038	관동화, 款冬花, Farfarae Flos	4	중국식물지, 77(1):93-4.	
TKM038	관동화, 款冬花, Farfarae Flos	5	本草學, 永林社	
TKM038	관동화, 款冬花, Farfarae Flos	6	대한약전+북한약전	
TKM038	관동화, 款冬花, Farfarae Flos	7	東醫四象新編. 四象人要藥	
TKM038	관동화, 款冬花, Farfarae Flos	8	한약재감별도감.식품의약품안전청	
TKM038	관동화, 款冬花, Farfarae Flos	9	동의보감	
TKM038	관동화, 款冬花, Farfarae Flos	10	조선민주주의인민공화국약전(제7판)	

oasis.html

tabulator활용하여 테이블로 화면 출력

```
var oasis_reference = {{oasis_reference|tojson}};
var oasis_reference_table = new Tabulator('#oasis_reference',{
    data:oasis_reference,
    pagination:"local",
    paginationSize:10,
    columns:[
        {title:"tkm_key", field:"tkm_key"},
        {title:"약재명", field:"약재명"},
        {title:"No", field:"No"},
        {title:"참고문헌", field:"참고문헌"},
        {title:"url", field:"url", width:300, formatter:"link", formatterParams:{target:"_blank"}},
    ],
});
```

oasis.html (내부 JS)

```
@app.get("/oasis")
async def root(request:Request, tkm_key:str):
    oasis_clinical, oasis_preclinical, oasis_reference = get_bio_everything.get_oasis(tkm_key)
    return templates.TemplateResponse("oasis.html", {"request":request, "oasis_clinical":oasis_
```

main.py

scientific_detail.html

질병명을 input으로 하여 medline의 summary와 annotation_table의 scientific detail과 reference 조회

go home

Overview | Scientific Detail

disease_trait_list : Asthma

summary : [{'content': "What is asthma?Asthma is a chronic (long-term) lung disease. It affects your airways, the tubes that carry air in and out of your lungs. When yo
up.What causes asthma?The exact cause of asthma is unknown. Genetics and your environment likely play a role in who gets asthma.An asthma attack can happen whe
Allergens are substances that cause an allergic reaction. They can includeDust mitesMoldPetsPollen from grass, trees, and weedsWaste from pests such as cockroaches a
smokeOccupational asthma is caused by breathing in chemicals or industrial dusts at workExercise-induced asthma happens during physical exercise, especially when th
of having asthma:Being exposed to secondhand smoke when your mother is pregnant with you or when you are a small childBeing exposed to certain substances at w
Puerto Ricans are at higher risk of asthma than people of other races or ethnicities.Having other diseases or conditions such as obesity and allergiesOften having viral r
tightnessCoughing, especially at night or early morningShortness of breathWheezing, which causes a whistling sound when you breathe outThese symptoms can range

scientific detail

DISEASE_TRAIT	SNPS	VARIANT AA CHANGE	GENENAME	var_type	EFFECT_ALLELE	ALLELE_1	ALLELE_2
Asthma	rs20541	p.Gln144Arg	IL13	missense_variant	G	A	G
Asthma	rs3894194	p.Arg18Gln	GSDMA	missense_variant	A	A	G
Asthma	rs2305479	p.Gly282Arg,p.Gly291Arg,p.Gly295Arg,p.Gly304Arg	GSDMB	missense_variant	T	C	T
Asthma	rs12123821			intron_variant	T	C	C
Asthma	rs4129267			intron_variant	T	C	C
Asthma	rs12123821			intron_variant	T	C	C
Asthma	rs903361			intron_variant	A	A	A
Asthma	rs7517302			intron_variant	T	T	T
Asthma	rs3771180			5_prime_UTR_variant	T	G	G
Asthma	rs1420101			synonymous_variant	T	C	C

ncbi.html

reference

LINK	AUTHOR	JOURNAL	STUDY
www.ncbi.nlm.nih.gov/pubmed/29273806	Demenais F	Nat Genet	Multiancestry association study identifies new asthma risk loci that colocalize with immune-cell enhancer
www.ncbi.nlm.nih.gov/pubmed/20860503	Moffatt MF	N Engl J Med	A large-scale, consortium-based genomewide association study of asthma.
www.ncbi.nlm.nih.gov/pubmed/30929738	Ferreira MAR	Am J Hum Genet	Genetic Architectures of Childhood- and Adult-Onset Asthma Are Partly Distinct.
www.ncbi.nlm.nih.gov/pubmed/21907864	Ferreira MA	Lancet	Identification of IL6R and chromosome 11q13.5 as risk loci for asthma.
www.ncbi.nlm.nih.gov/pubmed/31619474	Zhu Z	Eur Respir J	Shared Genetics of Asthma and Mental Health Disorders: A Large-Scale Genome-Wide Cross-Trait Analysi
www.ncbi.nlm.nih.gov/pubmed/21804548	Hirota T	Nat Genet	Genome-wide association study identifies three new susceptibility loci for adult asthma in the Japanese p
www.ncbi.nlm.nih.gov/pubmed/23028483	Ramasamy A	PLoS One	Genome-wide association studies of asthma in population-based cohorts confirm known and suggested l
www.ncbi.nlm.nih.gov/pubmed/17611496	Moffatt MF	Nature	Genetic variants regulating ORMDL3 expression contribute to the risk of childhood asthma.

scientific_detail.html

tabulator활용하여 테이블로 화면 출력

```
var reference = {{reference|tojson}};
var reference_table = new Tabulator('#reference',{
    data:reference,
    pagination:"local",
    paginationSize:10,
    columns:[
        // {formatter:"rowSelection", titleFormatter:"rowSelection", hozAlign:"center",
        // headerSort:false
        // },
        {title:"LINK", field:"LINK", width:300, formatter:"link", formatterParams:{target:"_blank"}},
        {title:"AUTHOR", field:"AUTHOR"},
        {title:"JOURNAL", field:"JOURNAL"},
        {title:"STUDY", field:"STUDY"},
    ],
});
```

scientific_detail.html (내부 JS)

```
@app.get("/scientific_detail_page")
async def root(request:Request, bio_input:str):
    summary = get_bio_everything.get_summary(bio_input)
    scientific_detail = get_bio_everything.get_scientific_detail(bio_input)
    reference = get_bio_everything.get_reference(bio_input)
    return templates.TemplateResponse("scientific_detail_page.html", {"request":request,
```

main.py

gene Summary

NCBI Gene Summary

Predicted to enable phosphatidylinositol-4,5-bisphosphate binding activity; phosphatidylinositol-4-phosphate binding activity; and phosphatidylserine binding activity.

ncbi.html

JS 안들어감 (only html화면 출력)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>GeneLinks_report</title>
    <link href="{{url_for('static',path='/style.css')}}" rel="stylesheet">
    <link href="https://unpkg.com/tabulator-tables@5.4.3/dist/css/tabulator_semanticui.min.css" rel="stylesheet">
</head>
<body>
    <p><strong>NCBI Gene Summary</strong></p>
    <p>{{ncbi_gene}}</p>
    <script type="text/javascript" src="https://unpkg.com/tabulator-tables/dist/js/tabulator.min.js"></script>
    <script src="{{url_for('static',path='/js/script.js')}}"></script>
</body>
</html>
```

ncbi.html

ncbi의 경우 input이 null일 경우 오류창을 띄우게 함 (그외 차이점 없음)

```
@app.get("/ncbi")
async def igsr(request:Request, genename:str):
    try:
        ncbi_gene = get_bio_everything.get_ncbi(genename)
        return templates.TemplateResponse("ncbi.html", {"request":request, "ncbi_gene":ncbi_gene})
    except:
        return f'genename is not exist'
```

main.py

5. docker활용 서버배포

개발서버

docker 설치 : https://www.docker.com/products/docker-desktop/
docker 실행 : service start docker
harbor 실행 : cd harbor || docker-compose up
image 빌드 : docker build -t [이미지이름] [폴더경로]
harbor형식으로 태그 :docker tag [이미지이름] [harbor url 경로 포함 이름]
harbor에 올리기 : docker push [harbor url 경로 포함 이름]

운영서버

docker 설치 : sudo apt install docker-ce docker-ce-cli containerd.io
docker 실행 : service start docker
harbor에서 image 가져오기 : docker pull [harbor에서 만든 이미지id]
image 실행 : docker run [harbor에서 만든 이미지id] -p 0.0.0.0:8000:8000

```
app
  __pycache__
  array_annotation
    __pycache__
    array_annotation.py
    connect_1kgp.sql
    connect_dbsnp.sql
    connect_gwas.sql
    connect_uniprot.sql
  vcf_annotation
    __pycache__
    connect_1kgp.sql
    connect_dbsnp.sql
    connect_gwas.sql
    connect_uniprot.sql
    model_221017.py
    pyvcf_221017.py
    vcf_annotation.py
  config.py
  get_bio_everything.py
  main.py
input_data
  genome_Petra_Hwang_back.txt
  genome_Petra_Hwang_v5_Full_20
  SRR062634.vcf
static
  js
    pages.js
    script.js
  style.css
templates
  disease_trait.html
  igsr.html
  index.html
  ncbi.html
  oasis.html
  overview_page.html
  scientific_detail_page.html
.gitattributes
.gitignore
dockerfile
requirements.txt
```

dockerfile

```
FROM python:3.10

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

COPY ./app /code/app

COPY ./input_data /code/input_data

COPY ./static /code/static

COPY ./templates /code/templates

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```